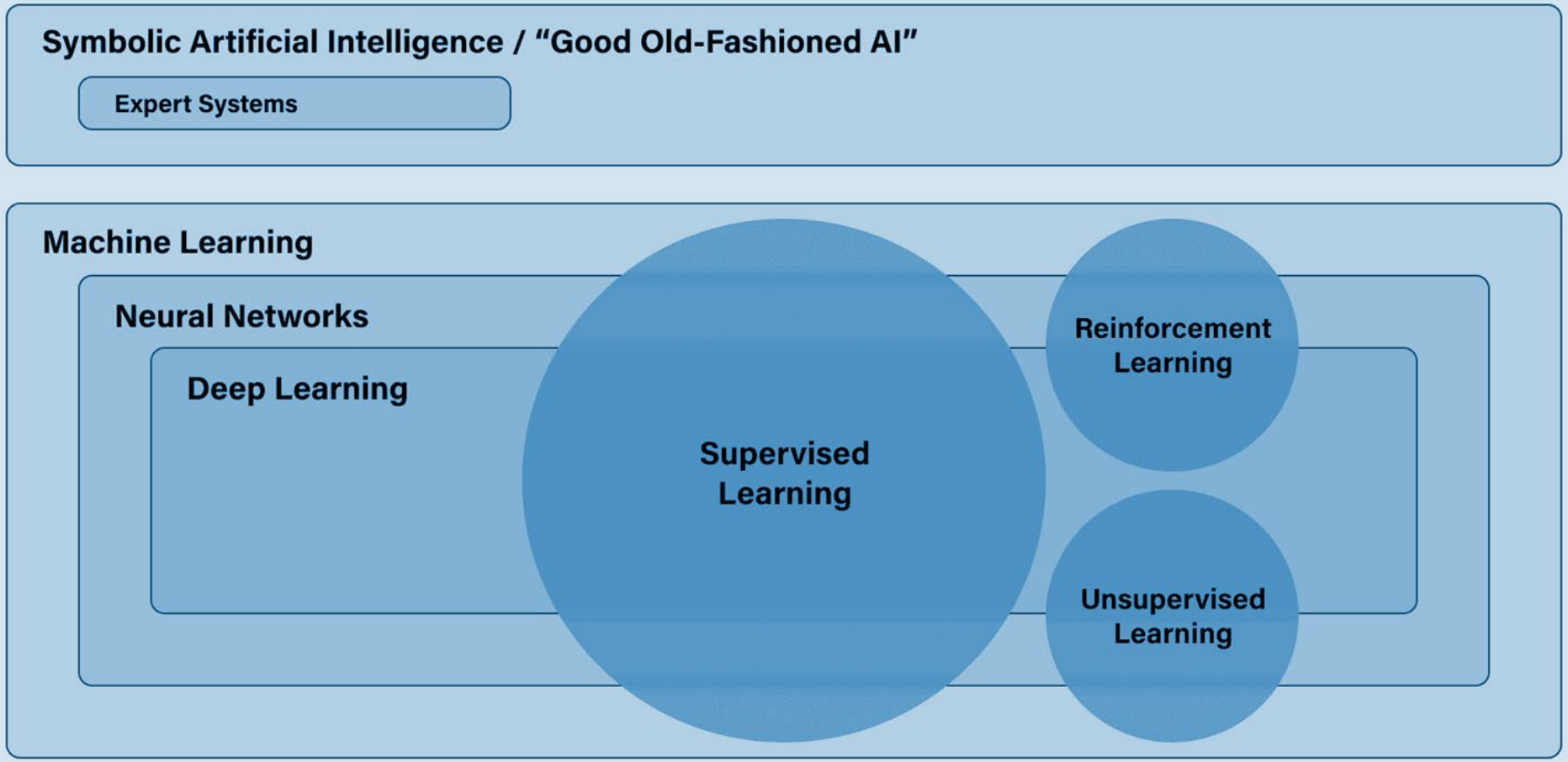
Redes neuronales

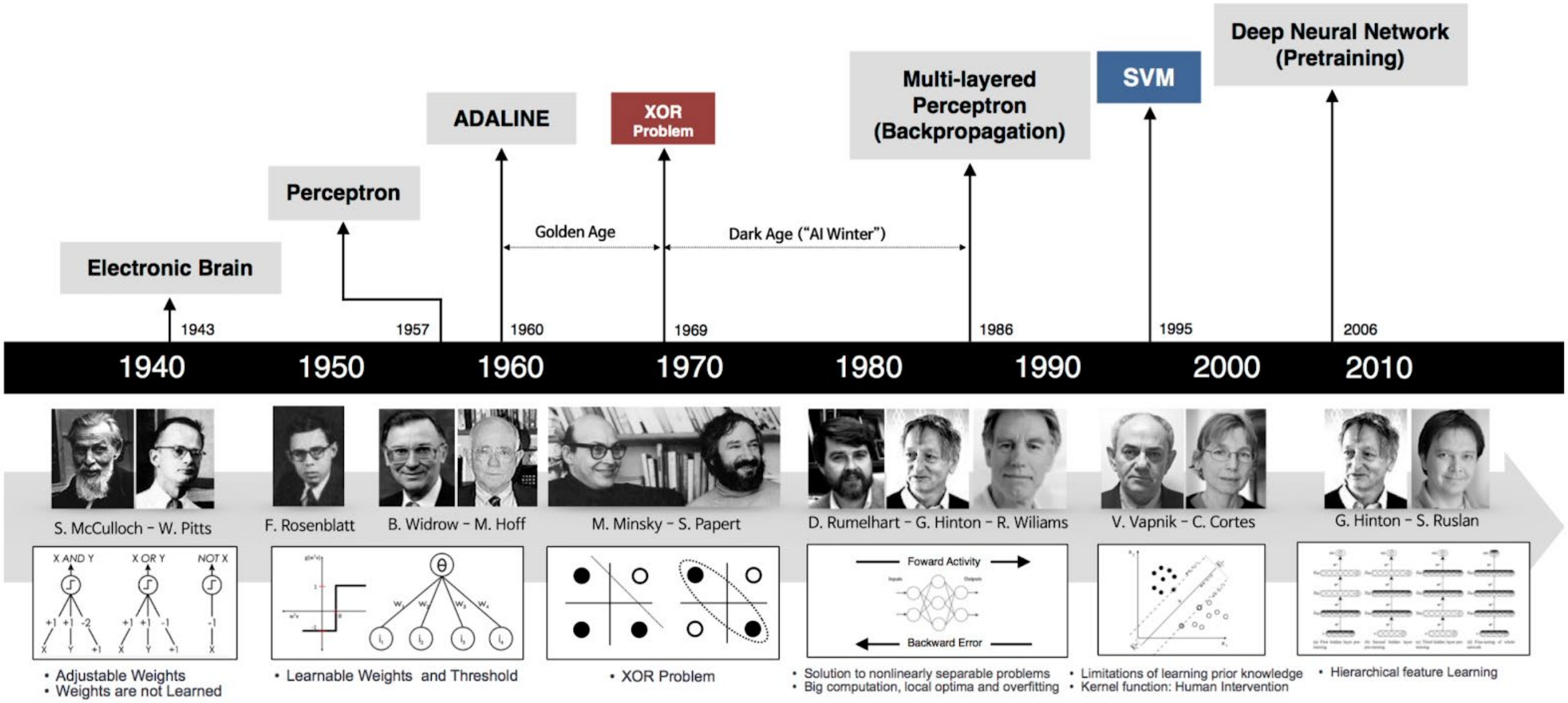
Artificial Intelligence



Recorrido histórico

1676 — Leibniz {discovers/invents) Chain Rule

Gauss & Legendre independently
invent linear regression (1805–1809)



- Adjustable Weights
- Weights are not Learned

- Learnable Weights and Threshold

- XOR Problem

- Solution to nonlinearly separable problems
- Big computation, local optima and overfitting

- Limitations of learning prior knowledge
- Kernel function: Human Intervention

- Hierarchical feature Learning

Conceptos técnicos sobre RRNN

Formato de Datos – Aprendizaje supervisado

Datatable

Features: *X*

target variable (o *label*): *y*

Entrenamiento:

Residence	Glucose	HbA1c	Smoking	Sex	BMI	**Diabetes Risk**
Urban	271	0.057	yes	1	210	**0.8**
Rural	115	0.489	no	2	190	**0.7**
Urban	252	0.057	no	2	215	**0.4**
Urban	123	0.059	no	2	200	**0.2**
Rural	9	0.07	no	1	240	**0.9**
Rural	124	0.064	yes	1	230	**0.6**
Rural	309	0.06	no	2	225	**0.9**
Urban	167	0.055	yes	1	220	**0.8**
Rural	12	0.071	no	1	220	**0.7**
Rural	10	0.062	no	2	215	**0.7**

→ Modelo

Predicción:

Residence	Glucose	HbA1c	Smoking	Sex	BMI
Urban	145	0.059	no	0	220

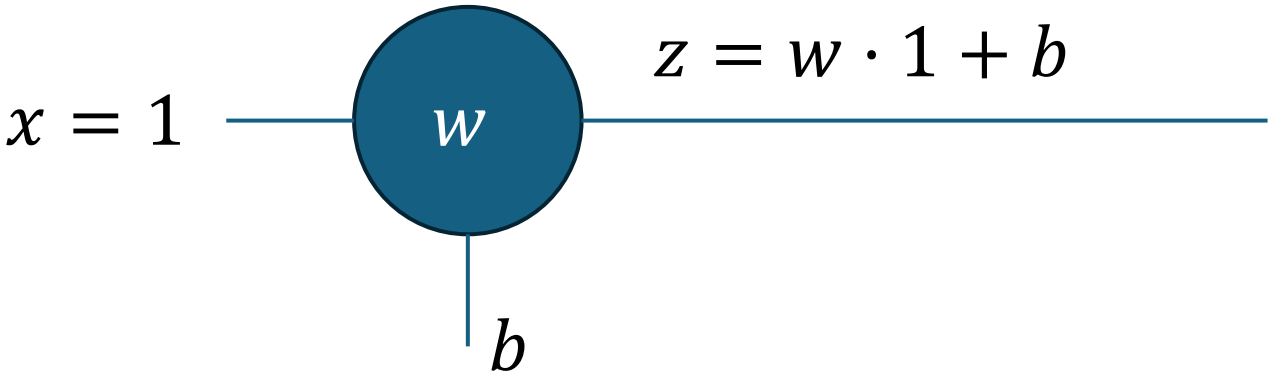
→ Modelo →

Diabetes Risk
0.7

Neurona: Un único input

Una neurona básica procesa una entrada escalar x y produce una salida activada a partir de un peso w y un sesgo b

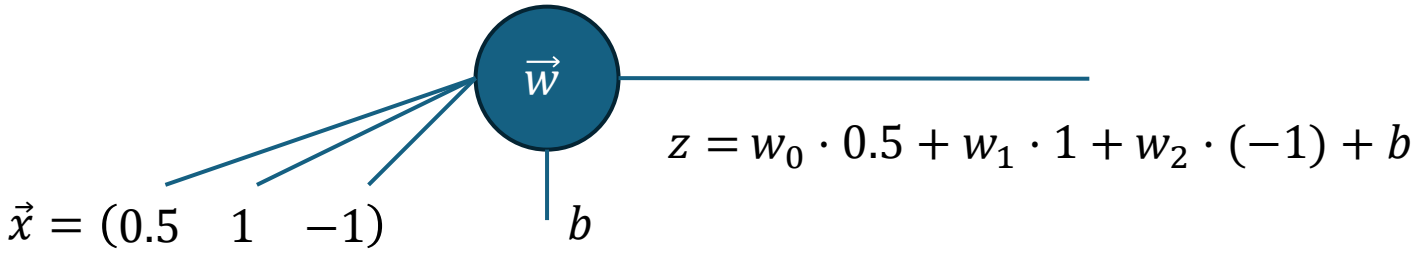
$$z = w \cdot x + b$$

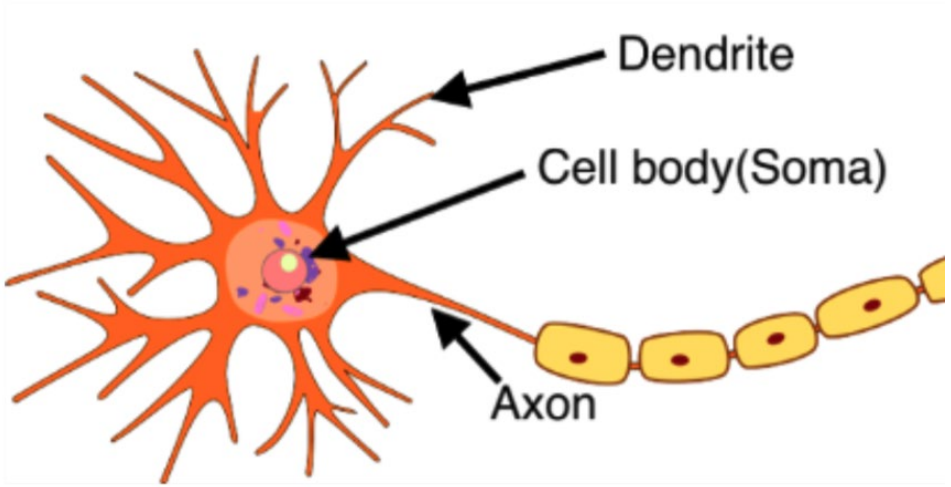


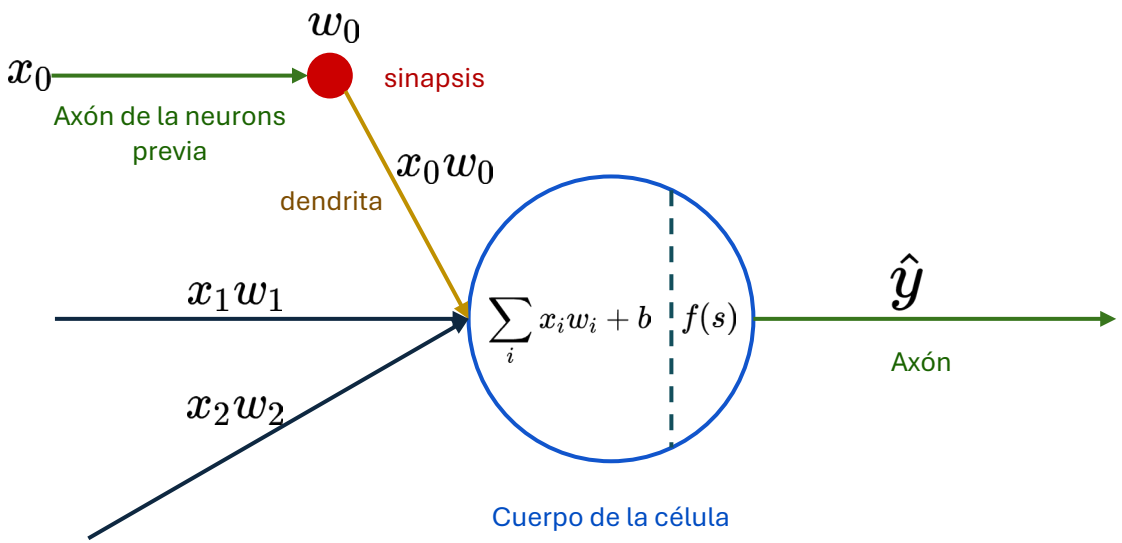
Neurona: Varios inputs

Con múltiples entradas x_1, x_2, …, la neurona realiza una combinación lineal de todas ellas y aplica la misma función de activación [Goodfellow et al., 2016]. {ecuación neurona multi-input}

$$z = w_0 \cdot x_0 + w_1 \cdot x_1 + w_2 \cdot x_2 + \cdots + b = \sum_j w_j \cdot x_j + b \ = \vec{w} \cdot \vec{x} + b$$

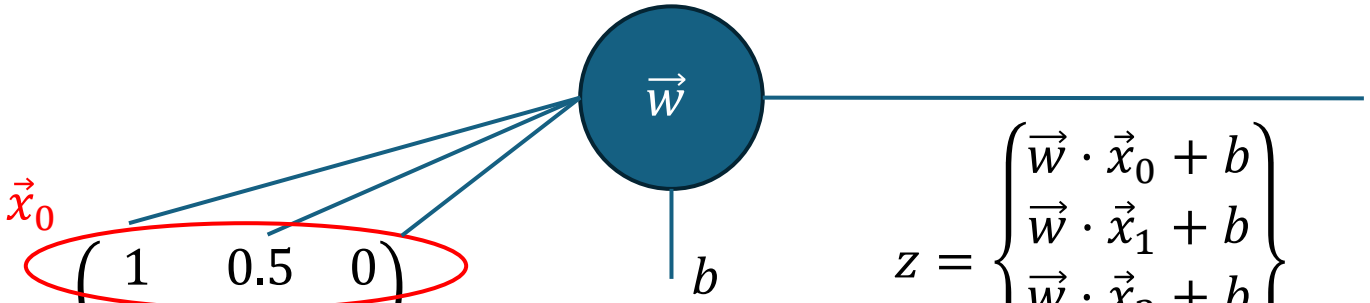



Dendrite
Cell body(Soma)
Axon


w_0
x_0
sinapsis
Axón de la neurons previa
$x_0 w_0$
dendrita
$x_1 w_1$
$x_2 w_2$
$\sum_i x_i w_i + b$
$f(s)$
$\hat{y}$
Axón
Cuerpo de la célula

Neurona: Batch data

Para acelerar el entrenamiento, las neuronas procesan lotes (batches) de muestras simultáneamente en matrices de datos, optimizando el uso de hardware.



$$\vec{x}_0$$

$$\boldsymbol{x} = \begin{Bmatrix} 1 & 0.5 & 0 \\ 0.8 & -1 & 1 \\ 0.6 & 3 & 1 \\ 0.2 & -1 & 1 \end{Bmatrix}$$

$$z = \begin{Bmatrix} \vec{w} \cdot \vec{x}_0 + b \\ \vec{w} \cdot \vec{x}_1 + b \\ \vec{w} \cdot \vec{x}_2 + b \\ \vec{w} \cdot \vec{x}_3 + b \end{Bmatrix}$$

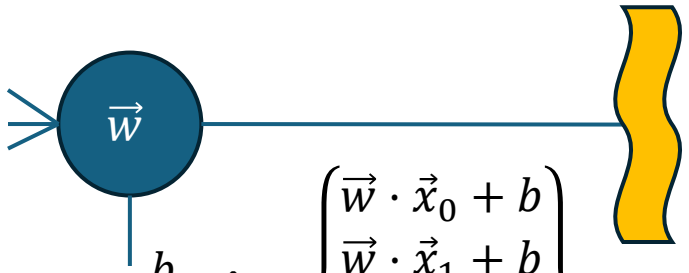
Función de pérdidas

Para problemas de regresión, la función de pérdidas que se usa (por defecto) es la MSE o RMSE

La función de pérdidas mide la discrepancia entre la salida predicha y la salida real, sirviendo de guía para ajustar los pesos.

$$\vec{x}_0$$

$$\boldsymbol{x} = \begin{Bmatrix} 1 & 0.5 & 0 \\ 0.8 & -1 & 1 \\ 0.6 & 3 & 1 \\ 0.2 & -1 & 1 \end{Bmatrix}$$



$$\hat{y} = \begin{Bmatrix} \vec{w} \cdot \vec{x}_0 + b \\ \vec{w} \cdot \vec{x}_1 + b \\ \vec{w} \cdot \vec{x}_2 + b \\ \vec{w} \cdot \vec{x}_3 + b \end{Bmatrix}$$

$$Loss = \sum_i (y_i - \hat{y}_i)^2$$

y: target variable, valores reales.

$\hat{y}$: valores predichos por el modelo

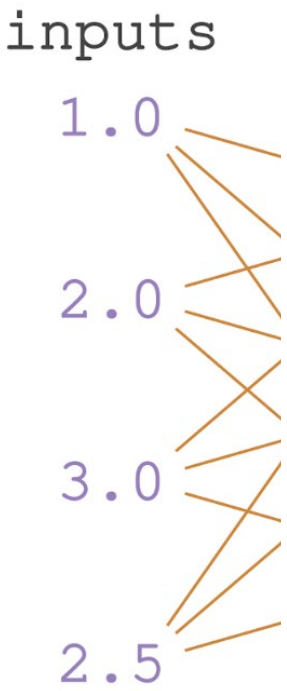
La función de pérdidas es una medida de "cuánto me estoy equivocando con los hiperparámetros que tengo ahora". Es la función a optimizar (minimizar).

Capa de entrada

Recibe directamente los datos, estableciendo la dimensionalidad de la información que la red manejará

$$\vec{x}_0$$

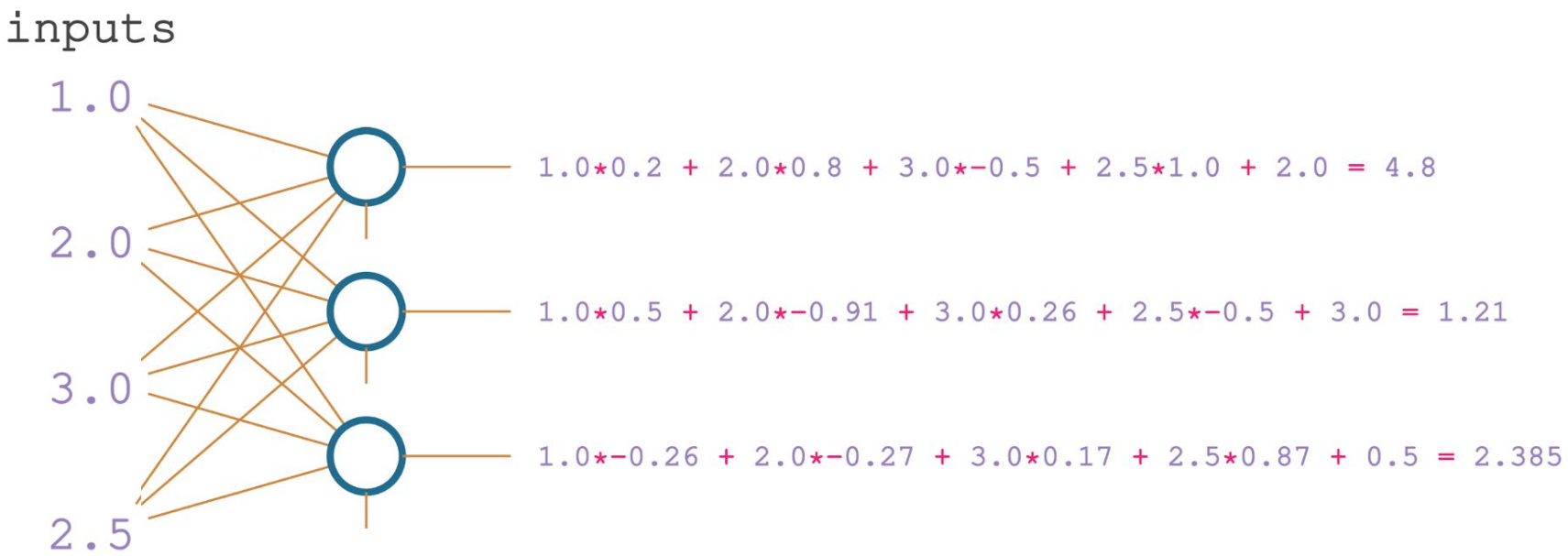
$$\boldsymbol{x} = \begin{Bmatrix} 1 & 2 & 3 & 2.5 \\ 0.8 & -1 & 1 & -0.5 \\ 0.6 & 3 & 1 & 4 \\ 0.2 & -1 & 1 & 0 \end{Bmatrix}$$



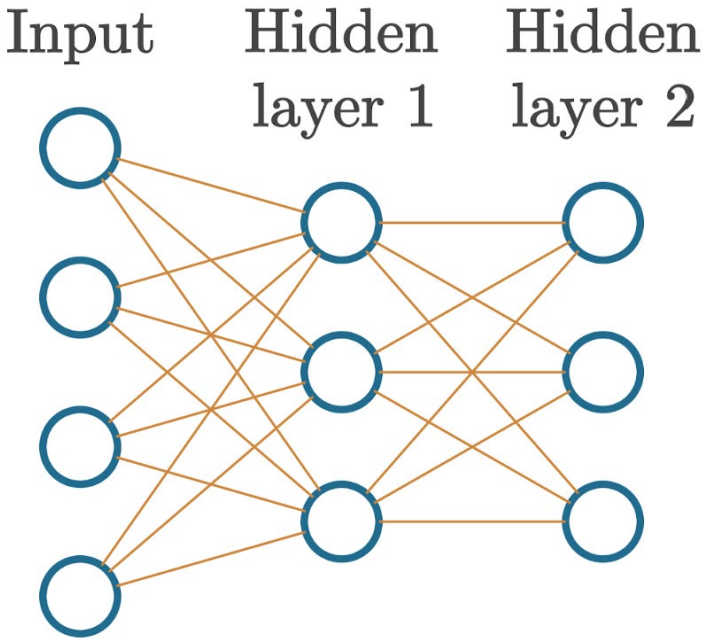
Capa de neuronas: Capa oculta

Aporta capacidad de modelado complejo al capturar características intermedias; su número y tamaño determinan la profundidad de la red.

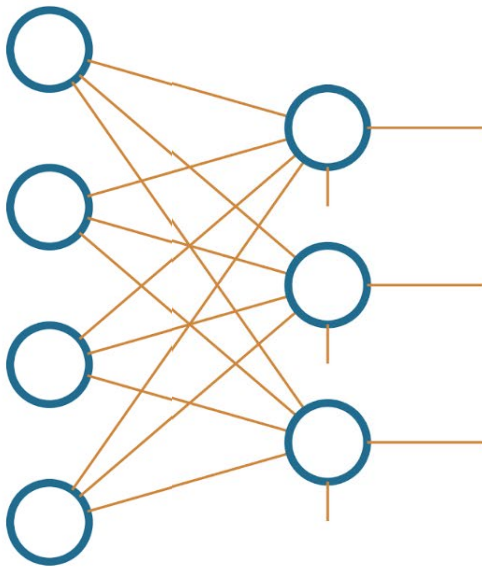
```
inputs = [1.0, 2.0, 3.0, 2.5]
weights1 = [0.2, 0.8, -0.5, 1.0]
weights2 = [0.5, -0.91, 0.26, -0.5]
weights3 = [-0.26, -0.27, 0.17, 0.87]
bias1 = 2.0
bias2 = 3.0
bias3 = 0.5
```



Capa de neuronas: añadiendo capas



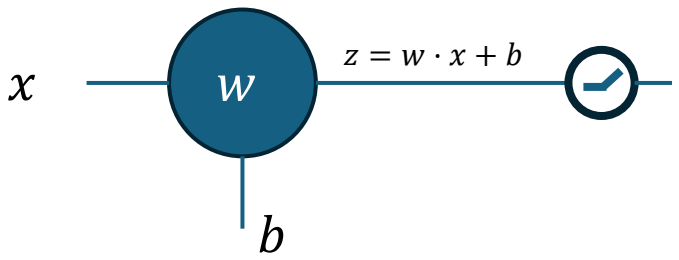
Capa de neuronas: última capa



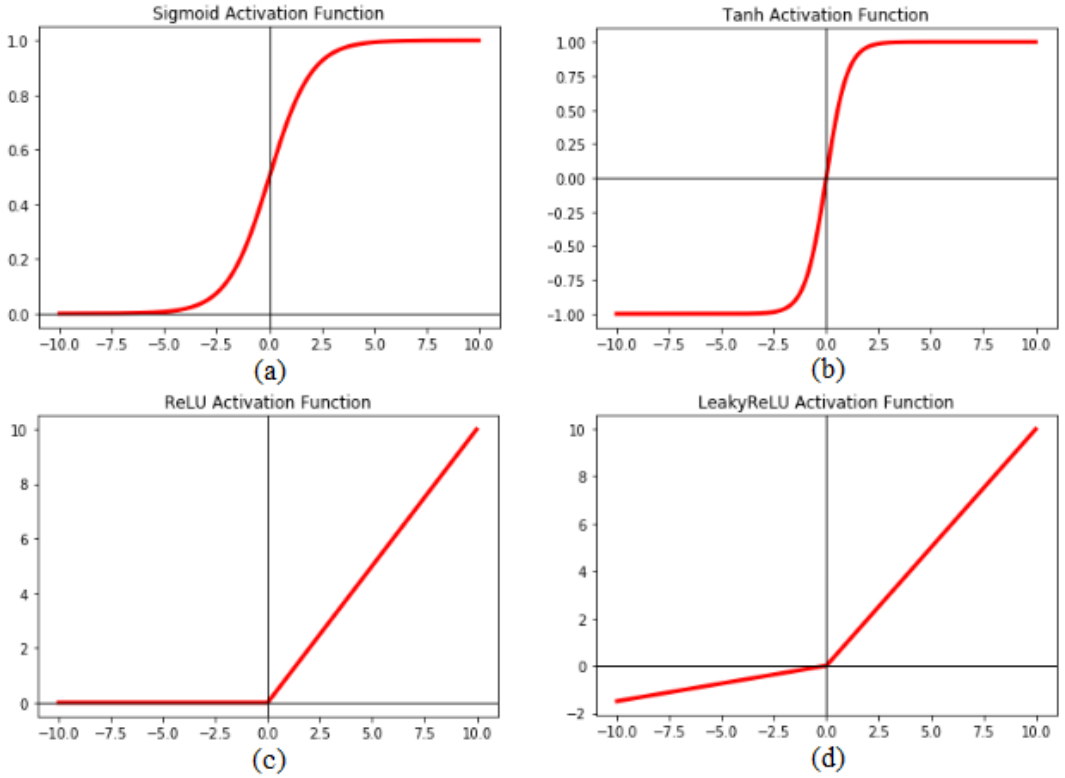
- En problemas de **clasificación** la última capa debe tener tantas neuronas como **clases** distintas haya
- En problemas de **regresión** la última capa debe tener tantas neuronas como **variables de salida** tenga el modelo

Función de activación

Introducen no linealidad y permiten que la red aprenda relaciones complejas; ejemplos frecuentes son ReLU, sigmoide y tanh.



$$a = \begin{cases} z & \text{if } z > 0 \\ 0 & else \end{cases}$$



Lo habitual es usar la misma función de activación para todas las capas ocultas, y una distinta dependiendo del problema para la capa de salida

Para problemas de <u>regresión</u>, no debe usarse ninguna función de activación en la última capa (o lo que es lo mismo, debe usarse la función lineal)

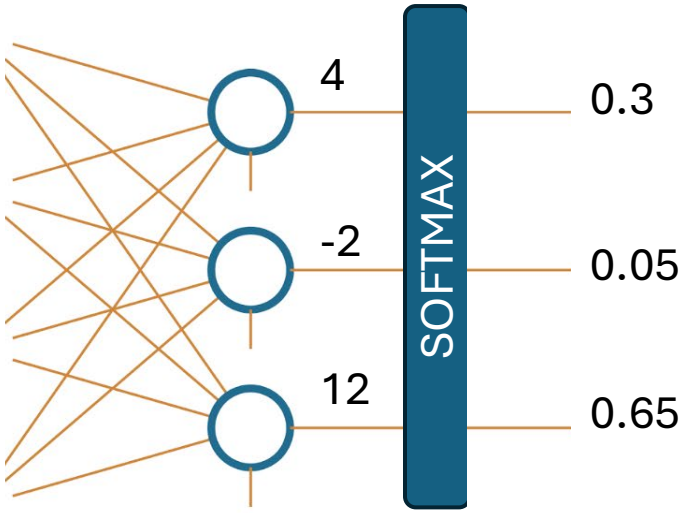
Clasificación: Función de activación Softmax

La función softmax transforma el vector z salida de la capa en una distribución de probabilidad sobre K clases.

$$\sigma(z)_i = \frac{e^{z_i}}{\sum_{j=1}^{K} e^{z_j}}$$

distribución de probabilidad ⇒

- Valores entre 0 y 1.
- Todos los valores suman 1

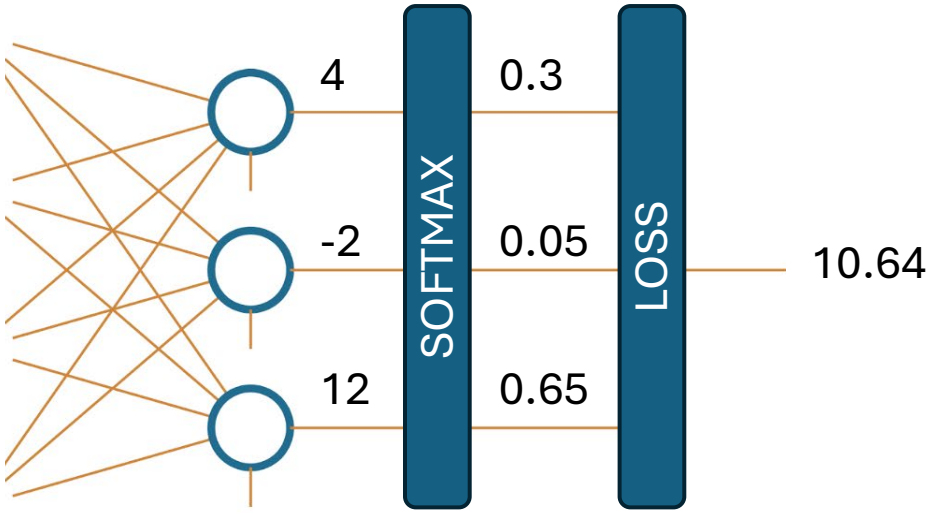


Importante: es la función de activación que debes escoger en la <u>última capa</u> para problemas de <u>clasificación</u>

Clasificación: Función de pérdidas Categorical Cross Entropy

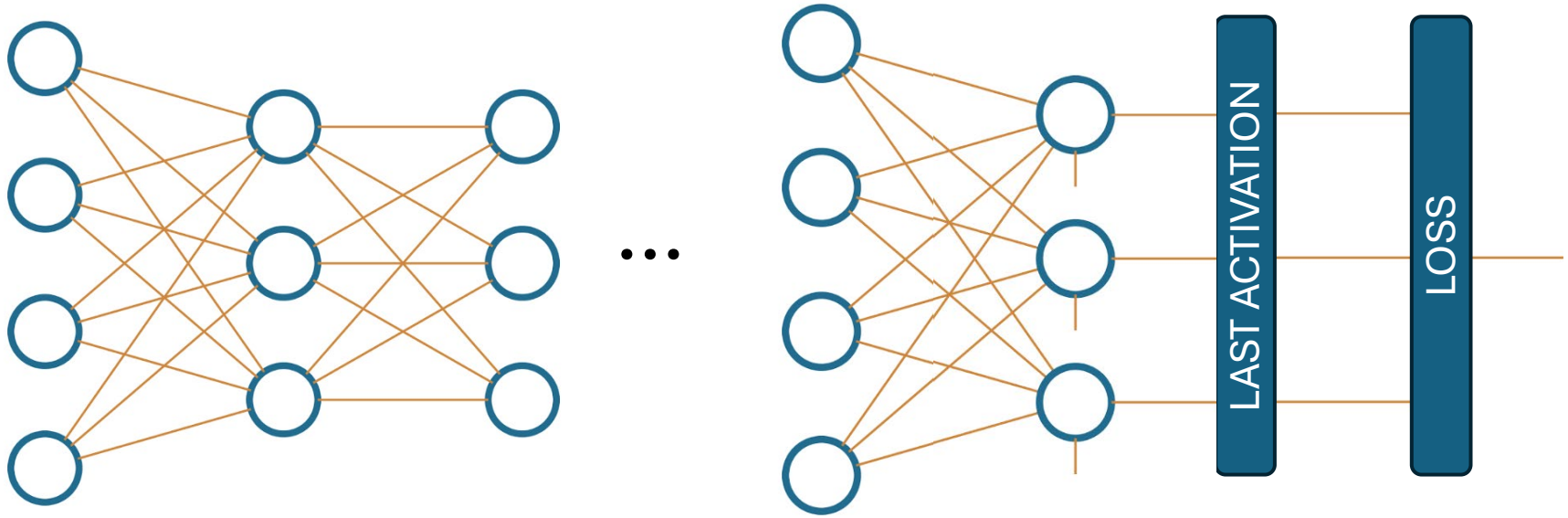
La función de pérdida de entropía cruzada mide la discrepancia entre la distribución de probabilidad real y la predicha

$$Loss \;\rightarrow\; L(y,\hat{y}) = -\sum_{i=1}^{K} y_i \cdot \log(\hat{y}_i)$$



Capa de neuronas: Arquitectura completa

Una red neuronal se compone de capas de entrada, ocultas y salida.

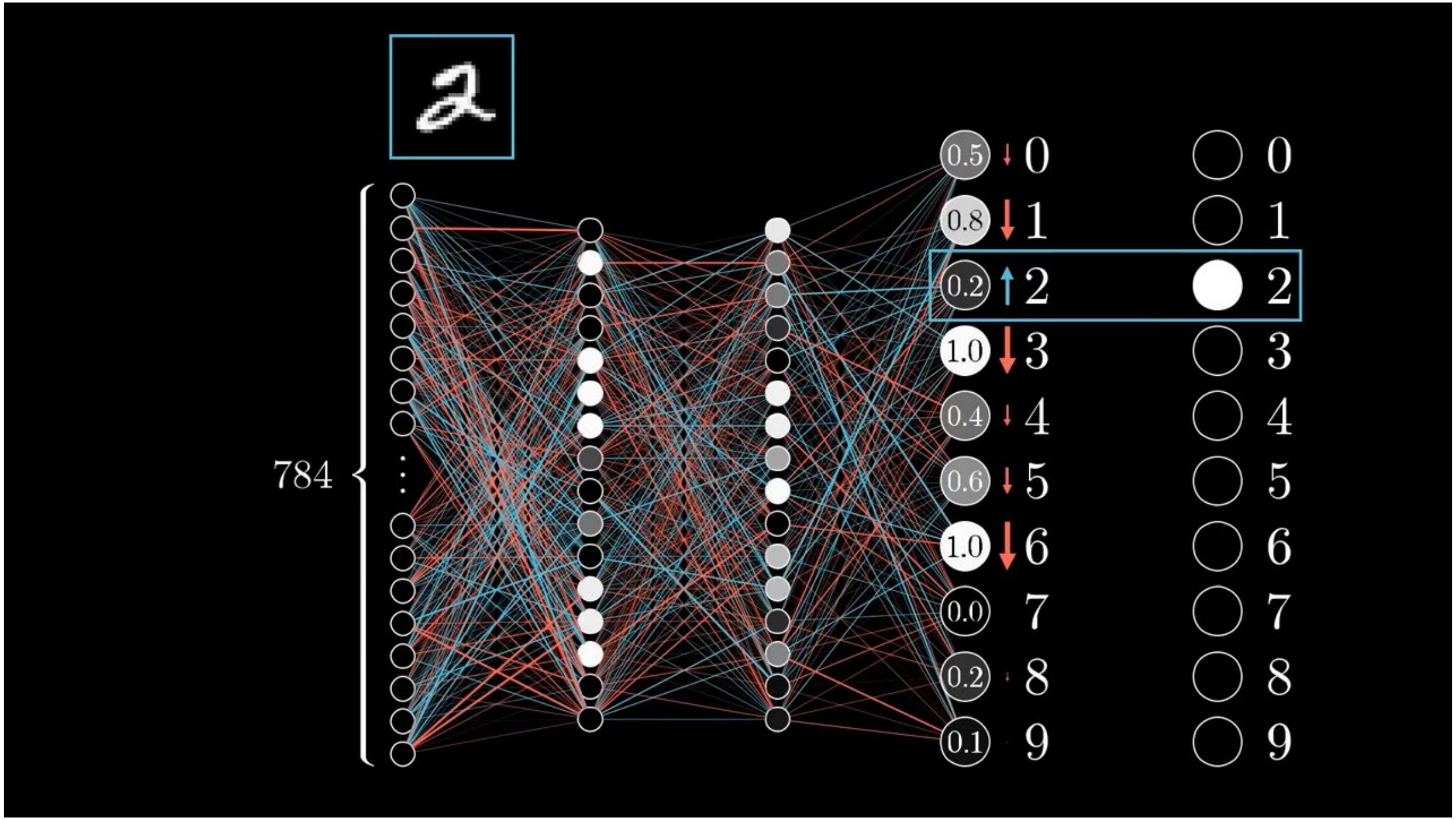


Teorema de aproximación universal

*Una red con al menos una capa oculta y funciones de activación no lineales puede aproximar **cualquier** función continua**

**suficientes datos, suficientes parámetros, bien ajustados*

Entonces, ¿cómo encontrar los valores correctos de todos los parámetros de la red?


784
0.5
0.8
0.2
1.0
0.4
0.6
1.0
0.0
0.2
0.1
0
1
2
3
4
5
6
7
8
9
0
1
2
3
4
5
6
7
8
9

Optimización - Descenso de gradiente

Estás en la cima de la colina y quieres ir al fondo

Optimización - Descenso de gradiente

Estás en la cima de la colina y quieres ir al fondo

1. Miras a tu alrededor y descubres la dirección que más se inclina hacia abajo.

Optimización - Descenso de gradiente

Estás en la cima de la colina y quieres ir al fondo

1. Miras a tu alrededor y descubres la dirección que más se inclina hacia abajo.
2. Das un paso en esa dirección.

Optimización - Descenso de gradiente

Estás en la cima de la colina y quieres ir al fondo

1. Miras a tu alrededor y descubres la dirección que más se inclina hacia abajo.
2. Das un paso en esa dirección.
3. Después de dar el paso, vuelves a comprobar dónde el suelo está más inclinado hacia abajo y das otro paso en esa dirección.

Optimización - Descenso de gradiente

Estás en la cima de la colina y quieres ir al fondo

1. Miras a tu alrededor y descubres la dirección que más se inclina hacia abajo.
2. Das un paso en esa dirección.
3. Después de dar el paso, vuelves a comprobar dónde el suelo está más inclinado hacia abajo y das otro paso en esa dirección.

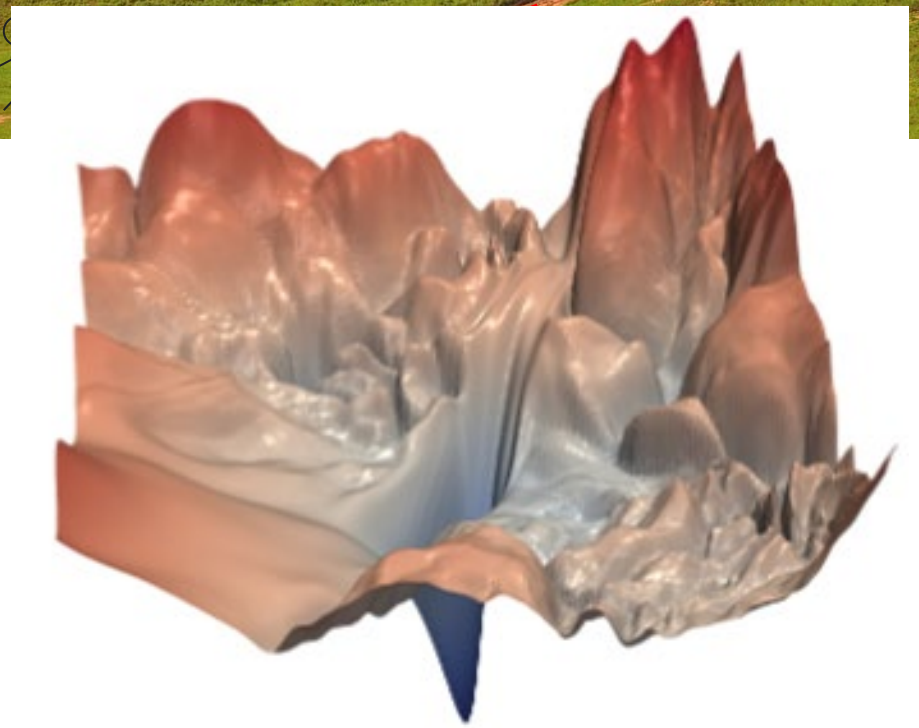
Sigues repitiendo esto hasta que llegas al fondo del valle.

Optimización - Descenso de gradiente

La colina = Función de pérdidas

Punto más bajo = mejor solución que minimiza la función de pérdidas

Proceso de buscar este mínimo = **Descenso de gradiente**

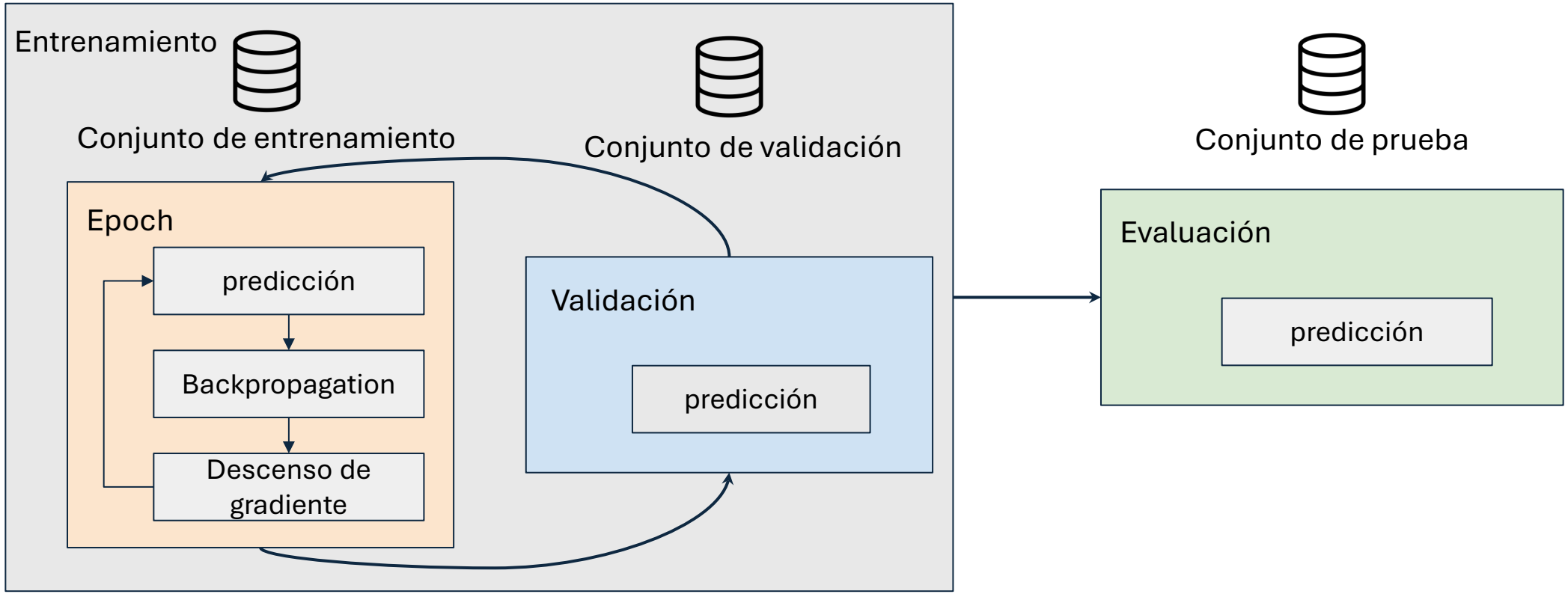


Deep Learning en la práctica

Entrenamiento: optimización del modelo

Validación: generalización de pruebas

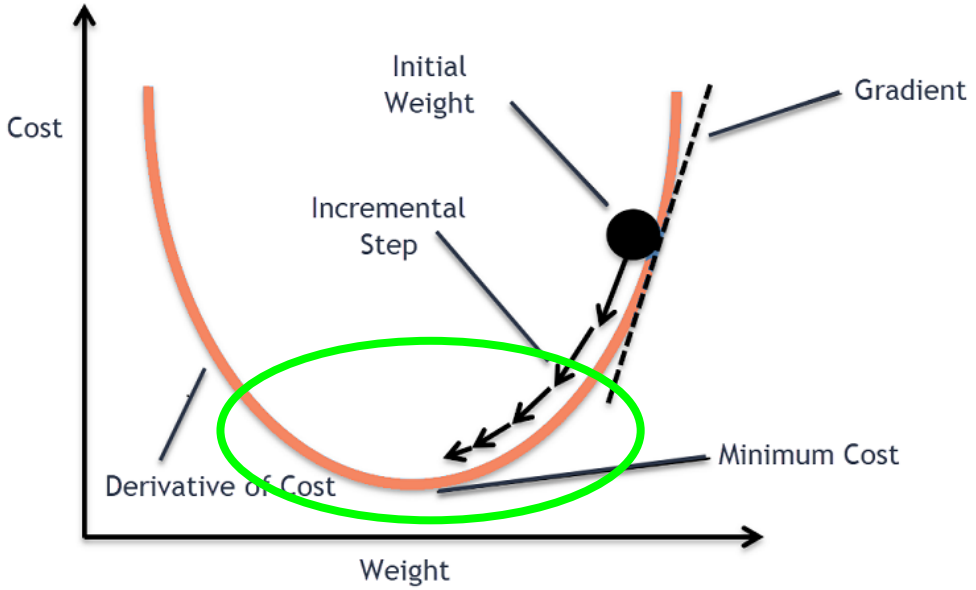
Evaluación: generalización de pruebas en otro conjunto de datos al final del entrenamiento

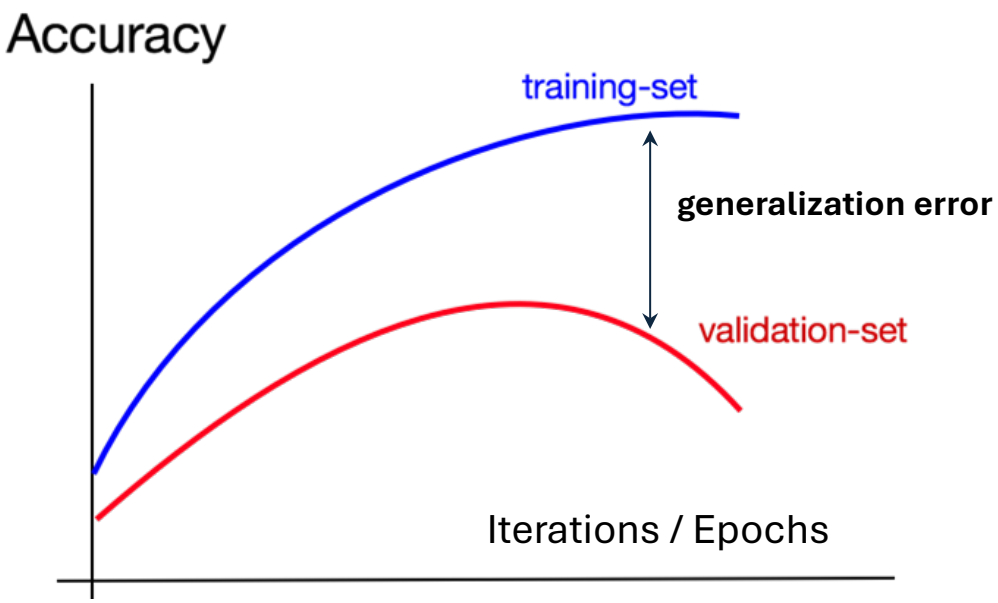


Combatiendo el overfitting

Regularización, dropout y early stopping son técnicas que reducen el sobreajuste mejorando la generalización.

- **Regularización**: Penaliza la complejidad del modelo añadiendo términos de regularización (L1, L2) en la función de pérdida, limitando pesos excesivos.
- **Dropout**: Durante el entrenamiento, desactiva aleatoriamente neuronas para evitar la dependencia excesiva en características específicas, promoviendo la robustez.
- **Early Stopping:** Finaliza el entrenamiento cuando la pérdida en validación deja de mejorar, evitando que el modelo se sobreajuste a los datos de entrenamiento.





Heurísticas (I): Tamaño de la red y otros

- **Capas y neuronas**: Más profundidad y ancho aumentan la capacidad, pero también el riesgo de sobreajuste y costo computacional. Comenzar con 1-3 capas ocultas, siendo una suficiente para problemas simples. Una regla común es usar 2^n neuronas, donde n es el número de features de entrada, y ajustar según el desempeño. Si la red no aprende, aumentar neuronas o capas; si sobreajusta, reducir la complejidad o aplicar regularización.
- **Activación**: ReLU por defecto
- **Regla empírica**: Para evitar sobreajuste, el número de parámetros P debería cumplir $P \leq N / 10$, donde N es el número de muestras de entrenamiento.

Heurísticas (II): Tamaño de la red y otros

- **Preproceso de datos:** en los ejercicios del curso ya viene dada la parte inicial de todo problema de ciencia de datos: la preparación de la base de datos. No es la parte central de este tema, pero es tan importante como el propio modelo. Alguna de las técnicas que aparecerán en los ejercicios y que son importantes para un correcto tratamiento de los datos son:
  - Tratamiento de datos perdidos (e.g. NaN) y anómalos (outliers). Eliminación de duplicados.
  - Codificación de variables categóricas
  - Tratamiento de bases de datos desbalanceadas
  - Normalizaciones, Transformaciones y escalados
  - Reducción de dimensionalidad